When I said this implementation was simple, I meant it: it uses a spin wait, so it's not ideal for cases where threads are likely to be waiting a long time, and it doesn't work if there's more than `count` threads that can potentially call `wait()` at any one time. If you need to handle either of those scenarios, you must use a more robust (but more complex) implementation instead. I've also stuck to sequentially consistent operations on the atomic variables, because that makes everything easier to reason about, but you could potentially relax some of the ordering constraints. Such global synchronization is expensive on massively parallel architectures, because the cache line holding the barrier state must be shuttled between all the processors involved (see the discussion of cache ping-pong in section 8.2.2), so you must take great care to ensure that this really is the best choice here.

Anyway, this is just what you need here; you have a fixed number of threads that need to run in a lockstep loop. Well, it's *almost* a fixed number of threads. As you may remember, the items at the beginning of the list acquire their final values after a couple of steps. This means that either you have to keep those threads looping until the entire range has been processed, or you need to allow your barrier to handle threads dropping out, and thus decreasing `count`. I opted for the latter option, because it avoids having threads doing unnecessary work just looping until the final step is done.

This means you have to change `count` to be an atomic variable, so you can update it from multiple threads without external synchronization:

```
std::atomic<unsigned> count;
```

The initialization remains the same, but now you have to explicitly `load()` from `count` when you reset the number of `spaces`:

```
spaces=count.load();
```

These are all the changes that you need on the `wait()` front; now you need a new member function to decrement `count`. Let's call it